AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Program drzewo BST
z zastosowaniem GitHub

Autor:
Michał Tokarz
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Cel techniczny – implementacja drzewa BST 3

1.2. Cel główny – praca zespołowa z użyciem systemu Git i platformy GitHub 3

1.3. Wizualizacja modelu rozgałęzień (Branching) 4

2. Analiza problemu **5**

2.1. Teoretyczne podstawy i zastosowanie drzewa BST 5

2.2. Sposób działania programu i mechanika pamięci 6

2.3. Ręczne śledzenie działania algorytmu (symulacja) 6

2.4. Proces powstawania kodu i wykorzystane narzędzia 7

3. Projektowanie **9**

4. Implementacja **10**

5. Wnioski **11**

Literatura **12**

Spis rysunków **12**

Spis tabel **13**

Spis listingów **14**

1. Ogólne określenie wymagań

1.1. Cel techniczny – implementacja drzewa BST

Zasadniczym celem technicznym projektu w ramach przedmiotu Programowanie zaawansowane (P2) jest zaprojektowanie i zaimplementowanie struktury danych w postaci Drzewa Poszukiwań Binarnych (ang. *Binary Search Tree* – BST). Program, napisany w języku C++ z wykorzystaniem kontenerów biblioteki standardowej (STL), ma obsługiwać pełen wachlarz operacji: dodawanie i usuwanie węzłów, całkowite czyszczenie drzewa, wyszukiwanie ścieżki do konkretnego elementu oraz tekstową wizualizację struktury (w porządkach preorder, inorder, postorder). Dodatkowo wymagana jest obsługa zapisu i odczytu stanu drzewa z wykorzystaniem plików tekstowych oraz binarnych, a całość ma być zarządzana przez interaktywne menu konsolowe.

1.2. Cel główny – praca zespołowa z użyciem systemu Git i platformy GitHub

Mimo rygorystycznych wymagań programistycznych, nadrzędnym celem dydaktycznym tego zadania jest opanowanie zasad pracy współdzielonej przy użyciu rozproszonego systemu kontroli wersji Git oraz platformy hostingowej GitHub. Projekt realizowany jest w dwuosobowym zespole, co wymusza zastosowanie profesjonalnych praktyk inżynierii oprogramowania (tzw. *collaborative workflow*).

Sprawne posługiwanie się systemem GitHub w zespole polega na równoległym rozwijaniu kodu bez wzajemnego nadpisywania swoich postępów. Podstawowym mechanizmem jest tu tworzenie osobnych gałęzi (ang. *branches*). Zamiast pisać kod bezpośrednio w gałęzi głównej (`main`), każdy z programistów odgałęzia nową ścieżkę w środowisku Visual Studio Code, implementuje przypisaną mu funkcjonalność, wykonuje serię zapisów historii (ang. *commits*), a następnie scala swoje zmiany z głównym projektem (ang. *merge*).

Zgodnie ze specyfikacją projektu, praca w zespole dwuosobowym przebiegała w następujących rygorystycznych fazach:

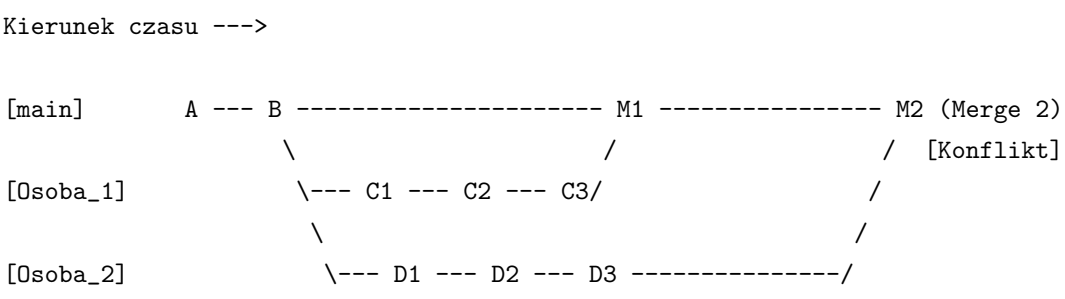
1. **Praca sekwencyjna (naprzemienna):** Pierwsza osoba tworzy własną gałąź, wykonuje na niej pracę (minimum 5 udokumentowanych commitów) i scala ją do `main`. Następnie druga osoba tworzy swoją gałąź na zaktualizowanym już kodzie, wykonuje swoje zadania i również przeprowadza proces scalania.

2. **Praca równoległa (współbieżna):** Obie osoby w tym samym czasie (z tego samego punktu startowego w historii) tworzą swoje niezależne gałęzie i rozpoczynają modyfikowanie tych samych plików kodowych.

3. **Rozwiązywanie konfliktów:** Próba jednoczesnego scalenia gałęzi z pracy równoległej celowo doprowadza do kolizji wersji (ang. *merge conflicts*). Zespół musi ręcznie rozstrzygnąć co najmniej 6 konfliktów (po 3 na każdą osobę), decydując za pomocą interfejsu Visual Studio Code, które bloki kodu pozostawić, które odrzucić, a które połączyć w nową całość.

1.3. Wizualizacja modelu rozgałęzień (Branching)

Poniższy uproszczony schemat obrazuje opisaną wyżej równoległą pracę dwuosobowego zespołu na odrębnych gałęziach, zakończoną procesem asynchronicznego scalania i koniecznością rozwiązywania konfliktów:



Jak widać na powyższym grafie, po wspólnym utworzeniu punktu bazowego **B**, obaj programiści rozpoczynają niezależną pracę na swoich gałęziach (commitując kolejno zmiany C i D). Osoba 1 szybciej kończy swoje zadanie i pomyślnie scala kod (punkt **M1**).

Kiedy Osoba 2 próbuje wypchnąć swoje modyfikacje na serwer i je scalić (punkt **M2**), system Git wykrywa, że pliki na gałęzi głównej zmieniły się w tzw. międzyczasie (w punkcie M1). Wymusza to na programistach bezpośrednią komunikację i manualne rozwiązanie konfliktów przed domknięciem procesu. Taki zorganizowany model gwarantuje absolutne bezpieczeństwo współdzielonego kodu, pozwalając unikać przypadkowego kasowania pracy drugiego członka zespołu.

2. Analiza problemu

2.1. Teoretyczne podstawy i zastosowanie drzewa BST

Drzewo Poszukiwań Binarnych (ang. *Binary Search Tree* – BST) to dynamiczna, nieliniowa struktura danych, która łączy w sobie zalety szybkiego wyszukiwania w posortowanej tablicy (wyszukiwanie binarne) z elastycznością alokacji pamięci znaną z list powiązanych. Każdy element drzewa, nazywany węzłem (ang. *node*), przechowuje określoną wartość (klucz) oraz maksymalnie dwa wskaźniki: na lewe i prawe poddrzewo.

Kluczową właściwością definiującą drzewo BST jest rygorystyczny porządek elementów: dla każdego węzła N wszystkie wartości w jego lewym poddrzewie muszą być od niego mniejsze, natomiast wszystkie wartości w prawym poddrzewie muszą być większe (lub równe, w zależności od wariantu implementacji). Dzięki tej własności, w optymalnym przypadku (gdy drzewo jest zbalansowane), operacje takie jak wyszukiwanie, dodawanie i usuwanie elementów osiągają logarytmiczną złożoność obliczeniową $O(\log n)$. W najgorszym jednak przypadku (np. przy wstawianiu danych posortowanych rosnąco), drzewo degeneruje się do postaci zwykłej listy (tzw. tyczka), a złożoność spada do liniowej $O(n)$.

Ze względu na unikalne cechy optymalizacyjne, algorytmy oparte na drzewach BST (oraz ich ulepszonych wariantach, takich jak drzewa AVL czy Red-Black Trees) znajdują bardzo szerokie zastosowanie w zaawansowanych rozwiązaniach informatycznych. Należą do nich:

- **Systemy bazodanowe (Indexowanie):** Mechanizmy baz danych (np. MySQL, PostgreSQL) wykorzystują struktury drzewiaste do tworzenia indeksów na tabelach, co pozwala na błyskawiczne odnajdywanie rekordów bez konieczności skanowania całych tabel (ang. *Full Table Scan*).

- **Systemy plików:** Zarządzanie hierarchią katalogów i szybkie lokalizowanie plików na dyskach operuje na drzewiastych algorytmach wyszukiwania.

- **Słowniki i pamięci podręczne:** Systemy autouzupełniania tekstu (np. w wyszukiwarkach internetowych) oraz sprawdzania pisowni bardzo często bazują na rozbudowanych wariantach drzew poszukiwań.

- **Tablice routingu:** W sieciach komputerowych algorytmy drzewiaste wspierają routery w szybkim dopasowywaniu adresów IP do odpowiednich ścieżek sieciowych.

2.2. Sposób działania programu i mechanika pamięci

Zaprojektowany program nie ogranicza się wyłącznie do struktur wskaźnikowych. Zgodnie ze specyfikacją, integruje on dynamiczne przydzielanie pamięci na stercie z nowoczesnymi kontenerami biblioteki standardowej (STL) języka C++ (takimi jak `std::vector` do gromadzenia wyników podczas przechodzenia drzewa oraz `std::string` do obsługi nazw plików).

Proces przechodzenia przez drzewo (ang. *Tree Traversal*) jest kluczowy do wyświetlania jego zawartości i zapisu na dysk. Aplikacja obsługuje trzy fundamentalne algorytmy przejścia:

- **Preorder (Wzdłużnie):** Odwiedź węzeł docelowy → przejdź lewe poddrzewo → przejdź prawe poddrzewo. Używany w projekcie do poprawnego serializowania (zapisu) struktury drzewa do pliku.
- **Inorder (Poprzecznie):** Przejdź lewe poddrzewo → odwiedź węzeł docelowy → przejdź prawe poddrzewo. Wynikiem zawsze jest ciąg elementów posortowanych rosnąco.
- **Postorder (Wstecznie):** Przejdź lewe poddrzewo → przejdź prawe poddrzewo → odwiedź węzeł docelowy. Niezbędny przy bezpiecznym, kaskadowym usuwaniu drzewa z pamięci (destruktor), by nie odciąć wskaźników do dzieci przed ich zwolnieniem.

Program wprowadza również zaawansowaną obsługę wejścia/wyjścia (I/O). Różnica pomiędzy zapisem tekstowym a binarnym polega na warstwie abstrakcji. Zapis tekstowy zapisuje znaki ASCII zrozumiałe dla człowieka, co zajmuje więcej miejsca na dysku. Zapis binarny (`std::ios::binary`) zapisuje bezpośrednio ciąg zer i jedynek na poziomie architektury procesora, co pozwala na błyskawiczny odczyt dużych pakietów pamięci do wektora STL bez żmudnego parsowania formatu tekstowego.

2.3. Ręczne śledzenie działania algorytmu (symulacja)

Aby zweryfikować teoretyczną poprawność algorytmu struktury BST, przeprowadźmy ręczną symulację wstawiania elementów oraz szukania drogi do wybranego węzła. Załóżmy, że do pustego drzewa wprowadzamy po kolei wartości: **50, 30, 70, 20, 40**.

1. **Krok 1 (Wstawienie 50):** Drzewo jest puste. Węzeł `[50]` staje się korzeniem.

2. **Krok 2 (Wstawienie 30):** Program porównuje `30` z `50`. $30 < 50$, więc schodzimy na lewe (puste) ramię. Węzeł `[30]` staje się lewym dzieckiem `50`.

3. **Krok 3 (Wstawienie 70):** $70 > 50$. Schodzimy na prawe ramię. Węzeł `[70]` staje się prawym dzieckiem korzenia.

4. **Krok 4 (Wstawienie 20):** $20 < 50$ (w lewo do `30`). Porównujemy z `30`: $20 < 30$. `[20]` staje się lewym dzieckiem `30`.

5. **Krok 5 (Wstawienie 40):** $40 < 50$ (w lewo do `30`). Porównujemy z `30`: $40 > 30$. `[40]` staje się prawym dzieckiem `30`.

Operacja wyszukiwania drogi do elementu (np. 40): Wywoływana jest funkcja `szukajDrogi(40)`. Program inicjalizuje wektor STL, w którym będzie zapisywał odwiedzane adresy/wartości.

- Start w korzeniu (`50`). Zapisujemy do ścieżki: `50`. Z racji $40 < 50$, wskaźnik przesuwa się w lewo.
- Obecny węzeł: `30`. Zapisujemy: `30`. $40 > 30$, wskaźnik przesuwa się w prawo.
- Obecny węzeł: `40`. Zapisujemy: `40`. Cel osiągnięty.
- Zwracany wynik (zapisany w wektorze): `50` $\rightarrow$ `30` $\rightarrow$ `40`. Złożoność wyszukiwania dla tej operacji wynosiła zaledwie 3 skoki, zamiast 5, jak w przypadku statycznej tablicy.

2.4. Proces powstawania kodu i wykorzystane narzędzia

Ponieważ projekt charakteryzuje się zaawansowaną architekturą dwuosobową (praca współbieżna), niezbędne stało się wdrożenie profesjonalnego ekosystemu narzędziowego. Każde z narzędzi pełniło w cyklu życia aplikacji (SDLC) ściśle określoną rolę:

- **System kontroli wersji Git:** Jest to silnik napędzający całą pracę zespołową. Śledzi on każdą modyfikację plików na poziomie bajtów (ang. *snapshots*). Pozwala na odizolowanie eksperymentalnych zmian dewelopera na osobnych gałęziach. Gdy następuje kolizja wersji – zjawisko pożądane i sztucznie sprowokowane w tym projekcie (np. gdy dwie osoby zedytowały plik `main.cpp` w tych samych linijkach) – Git blokuje proces *merge* (scalania) i wprowadza system znaczników konfliktów (`<<<<<<< HEAD`), wymuszając na programiście manualną interwencję i wybór poprawnego kodu.

- **Platforma GitHub:** Służy jako centralny węzeł synchronizacyjny dla obu członków zespołu. To w chmurze krzyżują się ścieżki (tzw. komendy `git push` oraz `git pull`), pozwalając na zdalną weryfikację zmian partnera (Code Review) przed zatwierdzeniem integracji.
- **Kontenery STL (`<vector>` oraz `<string>`):** Znacząco usprawniły zarządzanie blokami danych bez uciekania się do podatnych na błędy, niskopoziomowych tablic statycznych w języku C. Zapewniają automatyczną alokację ciągłą (realloc) pod maską.
- **Doxygen z silnikiem LaTeX:** Zapewnił ciągłość dokumentacyjną (ang. *Single Source of Truth*). Zamiast pisać osobną dokumentację opisującą parametry poszczególnych klas obsługi plików czy węzłów, specjalne komentarze zaszyte w plikach nagłówkowych są wyciągane przez parser i transformowane do formatu PDF.

3. Projektowanie

4. Implementacja

5. Wnioski

Spis rysunków

Spis tabel

Spis listingów